实验二 搜索算法

姓名：邱浩峰 学号：10245501462 2026 年 10 月 21 日

代码地址（GitHub，homework2 分支）：https://github.com/zechun115/qhf.git

一、实验完成情况

本次实验共四道题：前两题用 A* 求最短路径，第三题实现对弈树上的 Minimax 和 Alpha-Beta 剪枝，第四题把 A* 和博弈搜索结合起来做宝藏分配。四道题都用题面给的例子跑过，输出和预期一致，结果见表 1。程序入口是 main.py，运行后会把四道题的输出和预期值逐条比对，打印“通过”或“不通过”，方便检查。

题目	要求	运行结果	与预期
一　迷宫寻路	最短路径坐标	(0,0)→…→(3,4)，8 个点，代价 6	一致
二　加权地形	最小总代价	7	一致
三　博弈树	根节点得分	5	一致
四　宝藏分配 H=2	A 的首步与收益差	T2，-1	一致
四　宝藏分配 H=4	同上	T3，3	一致

表 1 四道题的运行结果

二、题目一：迷宫寻路

2.1 实现方法

用 A* 实现。open 表用 heapq 做小顶堆，按 f = g + h 排序；closed 用集合记录已经扩展过的格子；同一个格子可能被重复放进堆里，所以出堆时先看它是否已在 closed 中，是就直接丢掉，这样不必到堆里修改元素。邻居取上下左右四个方向，每走一步代价记 1，启发函数用曼哈顿距离。

```
将起点按 f 入堆，g[起点] = 0
while 堆不空:
    取出 f 最小的格子 n
    if n 已扩展过: 跳过
    标记 n 已扩展
    if n 是终点: 沿 parent 回溯输出路径，结束
    for n 的四个邻居 m:
        if g[n] + 代价(m) < g[m]: 更新 g 和 parent，把 m 入堆
```

2.2 运行结果

题面的 5×5 迷宫输出 (0,0)→(0,1)→(0,2)→(1,2)→(1,3)→(1,4)→(2,4)→(3,4)，一共 8 个点、7 步，代价 6，和预期一样。我另外试了一个起点被障碍围死的例子，程序返回空列表，没有报错。图 1 是 h=0 和用曼哈顿两种情况的搜索过程，格子里写的是第几个被扩展的格子。h=0 是从起点一圈一圈往外扩，障碍物附近扩散得很明显；用曼哈顿以后基本沿着一条斜线走到终点，扩展的格子少一些。

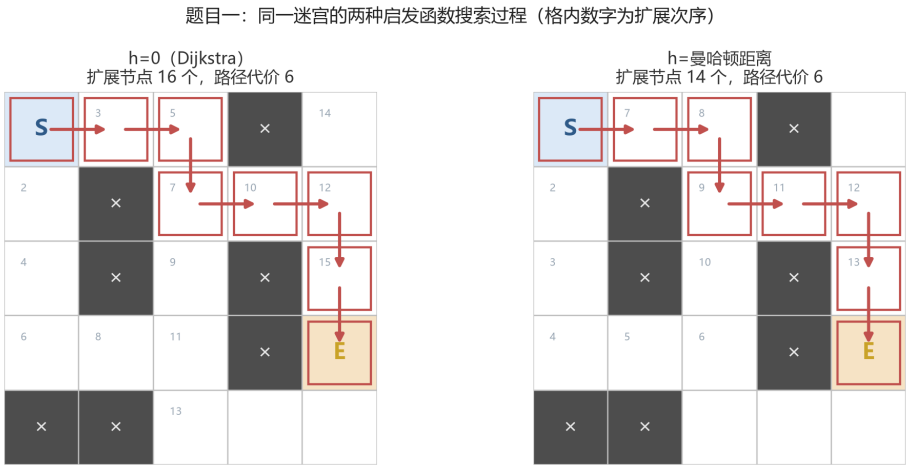


图 1 两种启发函数的搜索过程（格内数字为扩展次序，红色为最终路径）

2.3 不同启发函数的对比

启发函数	扩展节点	生成节点	路径代价
h = 0（Dijkstra）	16	16	6
曼哈顿距离	14	16	6
切比雪夫距离	12	15	6
欧氏距离（取整）	12	15	6

表 2 题面 5×5 迷宫上四种启发函数的开销

地图规模	h = 0	曼哈顿	切比雪夫	欧氏	曼哈顿占比	h=0 占比
21×21	307	83	221	215	19%	70%
41×41	1,174	279	907	882	17%	70%
61×61	2,638	430	2,144	2,105	12%	71%
81×81	4,642	981	3,933	3,881	15%	71%

表 3 随机迷宫（28% 障碍，每档 20 张）的平均扩展节点数

小地图上切比雪夫比曼哈顿还少扩展 2 个节点，一开始我以为程序写错了，把地图换大再测才明白是地图太小，个别格子的入堆顺序就能改变结果，样本太少说明不了问题。规模变大以后差距很明显：81×81 上曼哈顿只看了 15% 的可走格子，h=0 要看 71% 才能确定最短路径。原因是 h=0 相当于 Dijkstra，必须把代价小于最优解的区域全部扩完才敢停下。

另外我改了两个小地方。一是 f 相同时的取法：先出 h 小的（离终点更近），扩展节点是 981 个；按入队先后要 1,924 个，先出 h 大的要 2,000 个，只改一行排序键就差了一倍。二是把 f 改成 g + w • h：w=1.2 时扩展节点从 981 降到 380，路径只长了 2.4%；w=3 时只要 305 个节点，但代价多了 22.5%。所以不要求严格最短的时候，把 w 调大能快很多。

三、题目二：加权地形

3.1 运行结果

最优路径是 (0,0)→(1,0)→(2,0)→(3,0)→(3,1)→(3,2)→(4,2)→(4,3)→(4,4)，代价 7。这里我一开始算错了：把走进 E 的那一步也按格子上的数字算了，得出 9，和预期对不上。回头翻题面看到“最后一个格子没有代价”，把 E 的代价改成 0 以后就是 7。这条路线绕开了中间权值 8 和 3 的高代价区，宁可多走几步全是 1 的格子。

3.2 曼哈顿距离还能不能用

还能用。因为每走一步至少要花 1（E 记 0 只会让实际代价更小），所以真实的剩余代价不会小于曼哈顿距离，h 不会高估，A* 仍然返回最优解。

但好不好用是另一回事。权值 1~9 的 41×41 随机地图上，h=0 平均扩展 1,420 个节点，曼哈顿 1,413 个，几乎没有区别。因为实际走一步平均要花 5 左右，而 h 还是按每步 1 算，数值太小，f 里 g 占主导，排序效果和 Dijkstra 差不多。

3.3 给 h 乘一个系数

既然 h 偏小，就乘一个系数 k。这批地图最小权值是 3，理论上 k ≤ 3 时 h 仍然可采纳。

系数 k	平均扩展节点	代价 / 最优代价	次优解比例
0	1,426	1.0000	0%
1	1,424	1.0000	0%
3	1,401	1.0000	0%
4.5	699	1.0033	55%
6	111	1.0672	100%

表 4 k 取值对搜索开销和最优性的影响（41×41，20 张）

结果和理论对得上：k ≤ 3 时 20 张图全部是最优解；k=4.5 有一半以上是次优；k=6 全部次优，但节点数少了九成多。所以 k=3 就像一条线，越过去确实快，但是答案错了。

3.4 代价为 0 的格子会让 h 失效

如果地图里有代价 0 的格子（比如平地不花钱），上面“每步至少 1”就不成立了，曼哈顿会高估，A* 也就不能保证最优。我随机造了 60 张含 35% 零代价格的 15×15 地图，A* 有 28 张（46.7%）跑出来是次优解，平均差 2.5，最坏的一张最优 28、它给 36（差 8）。有点意外的是这一张 A* 只扩展了 83 个节点，比求出最优解的 Dijkstra（152 个）还少。原因是 h 把搜索引到了“看起来近”的地方，跑得快但是结果不对，所以不能只看搜索快慢判断算法好坏。

四、题目三：Alpha-Beta 剪枝

4.1 运行结果

题面那棵树 Minimax 返回 5，Alpha-Beta 也返回 5，说明剪枝没有改变结果。Minimax 要把 15 个节点全部访问、算 8 个叶子；Alpha-Beta 只访问 14 个节点、算 7 个叶子。图 2 是我按程序实际执行的顺序画的，被剪掉的分支用灰色虚线表示。具体过程是：根节点（MAX）先把左边子树算完，得到 α=5；左边那个 MIN 节点第一个孩子是 5，于是 β=5，它的第二个孩子（MAX）拿到窗口 (5, +∞)，算第一个叶子得 6，6 ≥ β=5，说明父节点已经有更好的 5 了，后面那个叶子 9 就不用看；右边 MIN 节点最后返回 2，2 ≤ α=5，但它是根的最后一个孩子，没有兄弟可剪了。

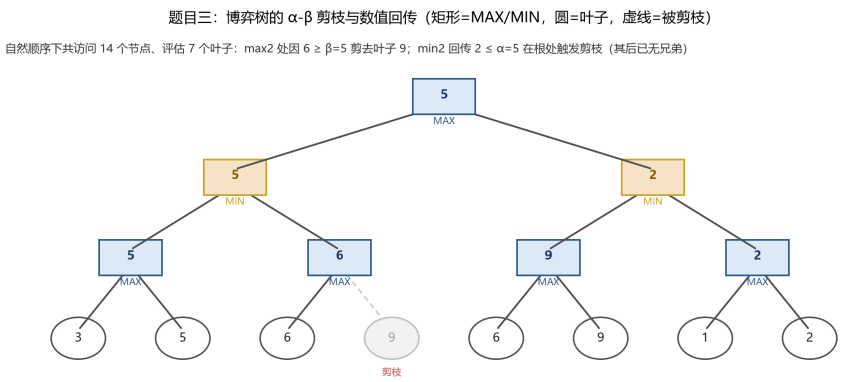


图 2 题面博弈树的实际搜索过程（矩形为 MAX/MIN 节点，灰圆为被剪掉的叶子）

4.2 子节点访问顺序的影响

访问顺序	自然顺序	最优优先	最差优先	随机
访问节点	14	11	15	11
评估叶子	7	5	8	5
根节点值	5	5	5	5

表 5 题面树上四种顺序的开销（根值都是 5）

访问顺序	自然顺序	最优优先	最差优先	随机	Minimax
平均访问节点	416	125	992	412	1,093
平均剪枝次数	96	46	150	94	—

表 6 随机树（分支 3、深度 6，60 棵平均）

从数据看有三点比较明显。第一，顺序影响很大：最优优先把节点从 1,093 压到 125，省了 88.6%，深度 7 的时候省 92.9%，差不多接近理论上 b 的 d/2 次方。第二，最差优先的剪枝次数最多（149.5 次），但访问的节点也最多（992 个）——它每次都在很深的地方剪掉很小的子树；最优优先只剪 46 次，可每次都在浅层剪掉一大片。所以剪枝要看剪在哪里，不是看剪了几次。第三，随机顺序和自然顺序几乎一样（412 和 416），说明不主动排序的话，Alpha-Beta 基本没什么用。

五、题目四：宝藏分配

5.1 距离矩阵

先用 A* 把 A、B、T1~T4 六个点两两之间的最小移动代价算出来，得到表 7。矩阵是对称的，也顺便验证了 A* 没写错。从表里能看出来：A 到 T2 只有 4，B 到 T3 只有 4，而 T4 离谁都远（B 到 T4 要 10，T1 到 T4 要 12），可它值 7 分。

起点＼终点	A	B	T1	T2	T3	T4
A	0	12	10	4	8	8
B	12	0	8	8	4	10
T1	10	8	0	6	6	12
T2	4	8	6	0	6	6
T3	8	4	6	6	0	6
T4	8	10	12	6	6	0

表 7 六个关键点之间的距离矩阵

5.2 求解结果

状态我用（A 的位置，B 的位置，还没被拿走的宝藏，A 的分数，B 的分数，轮到谁）来表示。轮到的人挑一个宝藏 Ti，收益是 V(i) 减去走过去的最小代价；到深度 H 或者宝藏拿完，就用 A 的分数减 B 的分数当评价值，A 要让它最大，B 要让它最小。

H=2 时 A 应该选 T2，收益差 -1；H=4 时选 T3，收益差 3，和预期一致，Minimax 和 Alpha-Beta 算出来完全一样。H=4 的走法是：A 拿 T3（收益 -1）→ B 拿 T1（+1）→ A 拿 T2（0）→ B 只能去 T4（-5），最后 (-1+0) - (1-5) = 3。

深度	首步 T1	首步 T2	首步 T3	首步 T4
H = 1	-1	+2	-1	-1
H = 2	-4	-1	-2	-4
H = 3	-4	+2	-1	-4
H = 4	-5	+1	+3	-7

表 8 以各宝藏作为首步时的收益差（正数表示对 A 有利）

看表 8 就明白为什么深度不同结论不同。只看一步（H=1）肯定是 T2，收益 +2；看到两步 T2 还是最好的（-1），因为别的首步更差；看到四步才知道应该先拿 T3——自己第一手亏 1 分，但把 B 的第二手逼到 T1→T4，让 B 亏 5 分，最后反而多 2 分。另外 H 是偶数时最后一手是 B 走，H 是奇数时最后一手是 A 自己走，所以不同深度的收益差不能直接比大小，只能在同一深度里比较。

5.3 一点优化

H=4 时一共要算 80 次收益，每次都重新跑一次 A* 要 5.6 ms。因为位置只会在 6 个点之间变化，我先把距离矩阵算好存起来，整个求解就降到 0.1 ms；再把子节点按收益从大到小排一下，扩展节点从 47 个减到 37 个。

六、遇到的问题和小结

- 题面细节要看清楚。第二题里 E 的代价记 0，我一开始漏看了，算出来是 9，和预期对不上。
- 启发函数要和代价的量级匹配。同样用曼哈顿，在单位代价的地图上是最好用的，到了权值 1~9 的地图上就退化成 Dijkstra；乘一个系数有用，但系数超过最小步长就不再最优了。
- Alpha-Beta 的效果几乎全看子节点顺序。排序好能省 90% 以上，排序差（最差优先）甚至比 Minimax 还慢，光加剪枝而不排序用处不大。
- 搜索深度会影响结论。浅层只能看到眼前的收益，深层才能发现“先吃小亏、让对手最后收烂摊子”这种走法；不同深度的评价值不能直接比较。
- 重复计算的东西提前算好很划算。题目四把距离矩阵预计算以后，耗时从 5.6 ms 降到 0.1 ms。

小结：这次实验代码量不大，但把两种搜索的基本做法都过了一遍，也做了一些对比实验。收获主要是明白了书上那几条结论（可采纳性、剪枝顺序、博弈深度）在实际数据上到底长什么样，以及写程序时容易在哪些细节上出错。

附：main.py 是四道题的入口；src/astar.py 是 A*，src/game_tree.py 是 Minimax 和 Alpha-Beta，src/treasure.py 是第四题；experiments/ 下面是实验脚本、结果 JSON 和插图。依次运行 main.py、run_experiments.py、make_figures.py、make_report_short.py 就能把结果、数据和这份报告重新生成一遍（随机种子固定，结果一致）。